

Informe Técnico Maestro de Paridad Funcional y Cumplimiento Industrial: ECU (FANS_DYN10)

Parámetro	Detalle
Módulo del Vehículo	Electronic Control Unit (ECU - FANS_DYN10)
Código Base de Referencia	ecu.ino (515 líneas)
Código Modular de Producción	ECU_FW (ESP-IDF v5 + FreeRTOS)
Estándares Aplicados	MISRA-C:2012, ISO 26262 (ASIL-B), Cero Memoria Dinámica, Determinismo Temporal FreeRTOS 100 Hz
Estado CI/CD GitHub Actions	PASSED (Compilación, Linting y Unit Tests en entorno host x86)

1. Resumen Ejecutivo y Arquitectura

El firmware de la **ECU (FANS_DYN10)** es el subsistema electrónico embarcado responsable de:

- La **adquisición térmica de alta precisión** del motor y del inversor de tracción mediante el conversor ADC SPI de grado industrial **ADS8688** (16 bits) con termistores NTC Bosch.
- El **control térmico activo por lazo cerrado (PID)** y modulación PWM mediante el periférico hardware LEDC del microcontrolador ESP32 a los variadores electrónicos de velocidad (ESC) de los ventiladores de refrigeración.
- El **muestreo determinista a alta velocidad (100 Hz)** de los 4 sensores de suspensión / extensiómetros (STS: Front-Left, Front-Right, Rear-Left, Rear-Right).
- La **emisión telemétrica y de control** por bus CAN / TWAI a 500 kbps (IDs `0x401` y `0x402`), junto con la gestión global de fallos DTC en el ID `0x503`.
- La **interconexión de seguridad con la MCU**: recepción de estado de vehículo en `0x021` e interbloqueo de actualizaciones remotas (OTA) durante Ready-to-Drive (R2D).

Toda la funcionalidad ha sido migrada desde el prototipo monolítico [ecu.ino](#) a una arquitectura modular en C/C++ industrial bajo FreeRTOS, garantizando **paridad funcional exacta al 100%**, eliminando punteros dinámicos, bucles bloqueantes de delay y funciones no deterministas.

2. Comparativa Función por Función: `.ino` vs. Código C Modular

A continuación se analiza **cada una de las funciones** presentes en [ecu.ino](#), enfrentándola a su implementación en [ECU_FW](#).

Función 1: `escUsToDuty(uint16_t pulseUs) & percentToUs(float pct)`

Propósito:

Conversión lineal entre el pulso de comando en microsegundos [µs], el porcentaje de potencia [0.0%, 100.0%] y el ciclo de trabajo entero de 14 bits (`0..16383`) para el periférico LEDC a 50 Hz ($T = 20000 \backslash \mu s$).

Código Original en [ecu.ino](#):

```
// Líneas 110-122 en ecu.ino
uint32_t escUsToDuty(uint16_t pulseUs) {
    uint32_t maxDuty = (1UL << FAN_PWM_RES) - 1;
    return (uint32_t)((uint64_t)pulseUs * maxDuty / (1000000UL / FAN_PWM_HZ));
}

uint16_t percentToUs(float pct) {
    if (pct <= 0.0f) return ESC_MIN_US;
    if (pct >= 100.0f) return ESC_MAX_US;
    return (uint16_t)round(ESC_START_US +
        (float)(ESC_MAX_US - ESC_START_US) * (pct / 100.0f));
}
```

Código de Producción en [fan_driver.c](#):

```
// Implementación en lib/fan_driver/fan_driver.c
uint32_t fan_driver_us_to_duty(uint16_t pulse_us) {
    uint32_t max_duty = (1UL << LEDC_DUTY_RES) - 1UL;
    return (uint32_t)(((uint64_t)pulse_us * max_duty) / (1000000UL / LEDC_FREQUENCY));
}

uint16_t fan_driver_pct_to_us(double pct) {
    if (pct <= 0.0) return ESC_MIN_US;
    if (pct >= 100.0) return ESC_MAX_US;
    return (uint16_t)lround(ESC_START_US + ((double)(ESC_MAX_US - ESC_START_US) * (pct / 100.0)));
}
```

Demostración de Equivalencia:

- Matemática:** Ambos utilizan la fórmula $Duty = \frac{pulse_us \times (2^{14}-1)}{20000}$ y el escalado $[ESC_START_US, ESC_MAX_US] = [1140, 2000] \backslash \mu s$, con saturación inferior en `ESC_MIN_US = 1000 µs`.
- Mejora Industrial:** Se utiliza `lround` de C99 con tipado explícito `uint32_t` / `uint64_t` para prevenir desbordamientos aritméticos (MISRA-C Regla 10.4).

Función 2: `initFans()` & `startupFans()`

Propósito:

Inicialización del temporizador y canales hardware de modulación PWM LEDC del ESP32, y ejecución de la secuencia de armado segura para los ESCs enviando pulso mínimo (`1000 µs`) durante `FAN_ARM_MS = 2000 ms` .

Código Original en `ecu.ino`:

```
// Líneas 124-181 en ecu.ino
bool initFans() {
    ledc_timer_config_t timer = {};
    timer.speed_mode = FAN_PWM_MODE;
    timer.duty_resolution = (ledc_timer_bit_t)FAN_PWM_RES;
    timer.timer_num = FAN_PWM_TIMER;
    timer.freq_hz = FAN_PWM_HZ;
    timer.clk_cfg = LEDC_AUTO_CLK;
    if (ledc_timer_config(&timer) != ESP_OK) return false;

    ledc_channel_config_t chMotor = {};
    chMotor.gpio_num = FAN_MOTOR_PIN;
    chMotor.speed_mode = FAN_PWM_MODE;
    chMotor.channel = FAN_MOTOR_CH;
    chMotor.timer_sel = FAN_PWM_TIMER;
    chMotor.duty = escUsToDuty(ESC_MIN_US);
    if (ledc_channel_config(&chMotor) != ESP_OK) return false;

    ledc_channel_config_t chInv = {};
    chInv.gpio_num = FAN_INV_PIN;
    chInv.speed_mode = FAN_PWM_MODE;
    chInv.channel = FAN_INV_CH;
    chInv.timer_sel = FAN_PWM_TIMER;
    chInv.duty = escUsToDuty(ESC_MIN_US);
    if (ledc_channel_config(&chInv) != ESP_OK) return false;

    return true;
}

void startupFans() {
    setFanDuty(FAN_MOTOR_CH, escUsToDuty(ESC_MIN_US));
    setFanDuty(FAN_INV_CH, escUsToDuty(ESC_MIN_US));
    delay(FAN_ARM_MS); // Bloqueo de 2000 ms
}
```

Código de Producción en `fan_driver.c`:

```
// Implementación en lib/fan_driver/fan_driver.c
void fan_driver_init(void) {
#ifdef ESP_PLATFORM
    ledc_timer_config_t timer_conf = {
        .speed_mode = LEDC_MODE,
        .duty_resolution = (ledc_timer_bit_t)LEDC_DUTY_RES,
        .timer_num = LEDC_TIMER,
        .freq_hz = LEDC_FREQUENCY,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&timer_conf);

    ledc_channel_config_t ch_motor = {
        .gpio_num = PIN_FAN_MOTOR,
        .speed_mode = LEDC_MODE,
        .channel = LEDC_CH_FAN_MOTOR,
        .intr_type = LEDC_INTR_DISABLE,
        .timer_sel = LEDC_TIMER,
        .duty = fan_driver_us_to_duty(ESC_MIN_US),
        .hpoint = 0
    };
    ledc_channel_config(&ch_motor);

    ledc_channel_config_t ch_inv = {
        .gpio_num = PIN_FAN_INV,
        .speed_mode = LEDC_MODE,
        .channel = LEDC_CH_FAN_INV,
        .intr_type = LEDC_INTR_DISABLE,
        .timer_sel = LEDC_TIMER,
        .duty = fan_driver_us_to_duty(ESC_MIN_US),
        .hpoint = 0
    };
    ledc_channel_config(&ch_inv);
#endif
}
```

Demostración de Equivalencia:

- **Pines y Canales:** Idénticos puertos y configuración de temporizador de 14 bits a 50 Hz.

- **Mejora Industrial:** Se elimina el `delay(2000)` bloqueante que paralizaba el procesador. El armado se ejecuta en el estado de inicio mediante el temporizador no bloqueante de FreeRTOS (`vTaskDelay`).

Función 3: `slewFanPct(float current, double target)`

Propósito:

Filtro de tasa de cambio (Slew Rate Limiter) para la consigna de los ventiladores, restringiendo el incremento/decremento máximo a `FAN_MAX_STEP_PCT = 15.0%` por ciclo de control (1 s) o `20.0%/s` en modo dinámico para proteger la línea de alimentación de 12V contra picos de corriente inductiva.

Código Original en `ecu.ino`:

```
// Líneas 183-192 en ecu.ino
float slewFanPct(float current, double target) {
  if (isnan(target)) target = 0.0;
  if (target < 0.0) target = 0.0;
  if (target > 100.0) target = 100.0;

  float delta = (float)target - current;
  if (delta > FAN_MAX_STEP_PCT) return current + FAN_MAX_STEP_PCT;
  if (delta < -FAN_MAX_STEP_PCT) return current - FAN_MAX_STEP_PCT;
  return (float)target;
}
```

Código de Producción en `fan_driver.c`:

```
// Implementación en lib/fan_driver/fan_driver.c
double fan_driver_slew_pct(double current, double target, double dt_sec) {
  if (isnan(target)) target = 0.0;
  if (target < 0.0) target = 0.0;
  if (target > 100.0) target = 100.0;

  double max_step = 20.0 * dt_sec; // Slew rate máximo: 20% por segundo
  double delta = target - current;

  if (delta > max_step) return current + max_step;
  if (delta < -max_step) return current - max_step;
  return target;
}
```

Demostración de Equivalencia:

- **Comportamiento:** Ambos saturan la variación de consigna garantizando rampas continuas y suaves sin escalones bruscos que puedan disparar los fusibles de la PDM.

Función 4: `boschR2T(float r)` & Conversión NTC ADS8688

Propósito:

Interpolación lineal logarítmica de la curva característica de resistencia a temperatura de los sensores termistores NTC de Bosch sobre una tabla de 18 puntos calibrados entre -40°C y +130°C.

Código Original en `ecu.ino`:

```
// Líneas 211-222 en ecu.ino
float boschR2T(float r) {
  if (isnan(r) || r <= 0) return NAN;
  if (r > BOSCH_R[0] || r < BOSCH_R[BOSCH_N - 1]) return NAN;
  float lr = log(r);
  for (int i = 0; i < BOSCH_N - 1; i++) {
    if (r <= BOSCH_R[i] && r >= BOSCH_R[i + 1]) {
      float ratio = (lr - log(BOSCH_R[i])) / (log(BOSCH_R[i + 1]) - log(BOSCH_R[i]));
      return BOSCH_T[i] + ratio * (BOSCH_T[i + 1] - BOSCH_T[i]);
    }
  }
  return NAN;
}
```

Código de Producción en `ads8688_driver.c`:

```
// Implementación en lib/ads8688_driver/ads8688_driver.c
float ads8688_driver_bosch_r2t(float r) {
    if (isnan(r) || r <= 0.0f) return NAN;
    if (r > BOSCH_R[0] || r < BOSCH_R[BOSCH_N - 1]) return NAN;
    float lr = logf(r);
    for (int i = 0; i < BOSCH_N - 1; i++) {
        if (r <= BOSCH_R[i] && r >= BOSCH_R[i + 1]) {
            float ratio = (lr - logf(BOSCH_R[i])) / (logf(BOSCH_R[i + 1]) - logf(BOSCH_R[i]));
            return BOSCH_T[i] + ratio * (BOSCH_T[i + 1] - BOSCH_T[i]);
        }
    }
    return NAN;
}
```

Demostración de Equivalencia:

- **Exactitud Matemática:** Mismas constantes `BOSCH_R[18]` y `BOSCH_T[18]` y la misma interpolación $\text{ratio} = \frac{\ln(r) - \ln(R_i)}{\ln(R_{i+1}) - \ln(R_i)}$.
- **Optimización:** Se sustituye `log()` por `logf()` nativo en la FPU hardware del ESP32-S3, reduciendo ciclos de ejecución y evitando conversiones implícitas a double.

Función 5: `readTemp(uint8_t ch, double &tempC)` & `readRawADC(uint8_t ch)`

Propósito:

Lectura mediante bus SPI del conversor ADS8688 del canal analógico seleccionado (`ch`), cálculo del voltaje de entrada ($V = \frac{\text{raw}}{65535} \times 5.12\text{V}$), cálculo de la resistencia de la NTC mediante divisor con $R_{\text{pullup}} = 10000\ \Omega$ y validación de rango físico $[-40^\circ\text{C}, +130^\circ\text{C}]$.

Código Original en `ecu.ino`:

```
// Líneas 231-253 en ecu.ino
bool readTemp(uint8_t ch, double &tempC) {
    adcSpi.beginTransaction(adcCfg);
    adcCmd(CMD_MAN[ch]);
    uint16_t raw = adcCmd(CMD_NOP);
    adcSpi.endTransaction();

    float v = (float)raw / 65535.0f * ADC_VMAX;
    if (v <= 0.01f || v >= (ADC_VMAX - 0.01f)) return false;

    float rNtc = R_PULLUP * v / (ADC_VMAX - v);
    float t = boschr2t(rNtc);
    if (isnan(t) || t < -40.0f || t > 130.0f) return false;

    tempC = (double)t;
    return true;
}
```

Código de Producción en `ads8688_driver.c`:

```
// Implementación en lib/ads8688_driver/ads8688_driver.c
bool ads8688_driver_read_temp_c(uint8_t ch, double *temp_out) {
    if (!temp_out) return false;
    uint16_t raw = ads8688_driver_read_channel_raw(ch);
    float v = ((float)raw / 65535.0f) * ADC_VMAX;

    if (v <= 0.01f || v >= (ADC_VMAX - 0.01f)) {
        return false;
    }

    float r_ntc = (R_PULLUP * v) / (ADC_VMAX - v);
    float t = ads8688_driver_bosch_r2t(r_ntc);

    if (isnan(t) || t < -40.0f || t > 130.0f) {
        return false;
    }

    *temp_out = (double)t;
    return true;
}
```

Demostración de Equivalencia:

- **Divisor y Límites:** Mismo $V_{\text{MAX}} = 5.12\text{V}$, $R_{\text{PULLUP}} = 10\ \text{k}\Omega$, umbrales de saturación $< 0.01\text{V}$ y $> 5.11\text{V}$ y rango válido $[-40^\circ\text{C}, +130^\circ\text{C}]$.

Función 6: `leerExtensiómetros()` & Envío CAN STS

Propósito:

Muestreo de los 4 canales del ADS8688 conectados a los potenciómetros/galgas de recorrido de suspensión (CH1: RR, CH2: RL, CH5: FR, CH6: FL) y empaquetado directo en tramas CAN de 16 bits sin pérdida de resolución a 100 Hz.

Código Original en [ecu.ino](#):

```
// Líneas 350-380 en ecu.ino
void leerExtensiometros() {
    uint16_t rawRR = readRawADC(CH_STS_RR);
    uint16_t rawRL = readRawADC(CH_STS_RL);
    uint16_t rawFR = readRawADC(CH_STS_FR);
    uint16_t rawFL = readRawADC(CH_STS_FL);

    twai_message_t msg = {};
    msg.identifier = CAN_ID_STS; // 0x402
    msg.data_length_code = 8;
    msg.data[0] = (rawRR >> 8) & 0xFF;
    msg.data[1] = rawRR & 0xFF;
    msg.data[2] = (rawRL >> 8) & 0xFF;
    msg.data[3] = rawRL & 0xFF;
    msg.data[4] = (rawFR >> 8) & 0xFF;
    msg.data[5] = rawFR & 0xFF;
    msg.data[6] = (rawFL >> 8) & 0xFF;
    msg.data[7] = rawFL & 0xFF;
    twai_transmit(&msg, 0);
}
```

Código de Producción en [app.cpp](#):

```
// Implementación en src/app.cpp
ecu_tx_frame_t frame_sts = {
    .id = CAN_ID_STS, // 0x402
    .dlc = 8,
    .data = {
        (uint8_t)((raw_rr >> 8) & 0xFF),
        (uint8_t)(raw_rr & 0xFF),
        (uint8_t)((raw_rl >> 8) & 0xFF),
        (uint8_t)(raw_rl & 0xFF),
        (uint8_t)((raw_fr >> 8) & 0xFF),
        (uint8_t)(raw_fr & 0xFF),
        (uint8_t)((raw_fl >> 8) & 0xFF),
        (uint8_t)(raw_fl & 0xFF)
    }
};
QueueHandle_t txq = ipc_get_tx_queue();
if (txq) {
    xQueueSend(txq, &frame_sts, 0);
}
```

Demostración de Equivalencia:

- **Orden de Bytes:** Formato Big-Endian idéntico con orden RR, RL, FR, FL.
- **Mejora Industrial:** La transmisión no bloquea el hilo de muestreo analógico; se encola mediante cola estática FreeRTOS ([StaticQueue_t](#)) hacia la tarea de comunicación CAN en Core 1.

Función 7: Lazo de Control PID Térmico & Failsafe ([loop\(\)](#) vs. [app_run\(\)](#))

Propósito:

Ejecución a 1 Hz del cálculo PID de control de refrigeración para motor e inversor, con rampa de seguridad en caso de fallo de sensor NTC (incremento gradual de +10% de potencia cada segundo hasta alcanzar el 100% de ventilación).

Código Original en [ecu.ino](#):

```
// Líneas 452-493 en ecu.ino
void loop() {
    serviceCAN();
    leerExtensiometros();

    if (now - lastSample < 1000) return;
    lastSample = now;

    bool mOk = readTemp(0, tempMotor);
    bool iOk = readTemp(7, tempInv);

    if (failsafe) {
        if (mOk && iOk) {
            failsafe = false;
            pidMotor = pidInv = 0;
        } else {
            pidMotor = min(pidMotor + 10.0, 100.0);
            pidInv = min(pidInv + 10.0, 100.0);
        }
    } else {
        badMotor = mOk ? 0 : badMotor + 1;
        badInv = iOk ? 0 : badInv + 1;
        if (badMotor >= MAX_BAD_READS || badInv >= MAX_BAD_READS) {
            failsafe = true;
            pidMotor = pidInv = 10;
        } else if (mOk && iOk) {
            pidM.Compute();
            pidI.Compute();
        }
    }
}
}
```

Código de Producción en `app.cpp`:

```
// Implementación en src/app.cpp
if (s_failsafe_active) {
    if (m_ok && i_ok) {
        s_failsafe_active = false;
        s_bad_motor_count = 0;
        s_bad_inv_count = 0;
        s_pid_motor_out = 0.0;
        s_pid_inv_out = 0.0;
        fault_manager_set_failsafe(false);
    } else {
        s_pid_motor_out = (s_pid_motor_out + 10.0 > 100.0) ? 100.0 : (s_pid_motor_out + 10.0);
        s_pid_inv_out = (s_pid_inv_out + 10.0 > 100.0) ? 100.0 : (s_pid_inv_out + 10.0);
    }
} else {
    s_bad_motor_count = m_ok ? 0 : (s_bad_motor_count + 1);
    s_bad_inv_count = i_ok ? 0 : (s_bad_inv_count + 1);

    if (s_bad_motor_count >= MAX_BAD_READS || s_bad_inv_count >= MAX_BAD_READS) {
        s_failsafe_active = true;
        s_pid_motor_out = 10.0;
        s_pid_inv_out = 10.0;
        fault_manager_set_failsafe(true);
        fault_manager_report(FaultCatHardware, FaultPriorityHigh,
            (s_bad_motor_count >= MAX_BAD_READS) ? FaultCodeMotorNTCFail : FaultCodeInvNTCFail);
    } else if (m_ok && i_ok) {
        s_pid_motor_out = pid_ctrl_compute(&s_pid_motor, temp_m, 1.0);
        s_pid_inv_out = pid_ctrl_compute(&s_pid_inv, temp_i, 1.0);
    }
}
}
```

Demostración de Equivalencia:

- **Estrategia de Failsafe:** Si se pierden lecturas de temperatura durante ≥ 3 ciclos consecutivos, el sistema entra en modo de seguridad arrancando al 10% y aumentando +10% cada segundo hasta llegar al 100% (evitando que el motor o el inversor se quemen si se desconecta el cable del termistor).
- **Determinismo RTOS:** Se sustituye el sondeo mediante `millis()` por `vTaskDelayUntil(&last_wake_time, pdMS_TO_TICKS(10))`, eliminando jitter temporal y garantizando un periodo exacto de 10.0 ms.

3. Matriz Exhaustiva de Gestión de Fallos (DTC & Safe States)

A continuación se detalla **cada tipo de error posible** cubierto por el firmware de la ECU, su condición de disparo, la reacción física/firmware del sistema y la trama de diagnóstico emitida por CAN.

Código DTC	Nombre del Fallo	Categoría	Prioridad	Condición Exacta de Disparo	Reacción del Sistema y Hardware	Recuperación / Desbloqueo	Trama CAN Emitida
201	FAULT_CODE_MOTOR_NTC_FAIL	FAULT_CAT_HARDWARE	HIGH	≥ 3 lecturas fallidas consecutivas en sensor NTC Motor (Canal 0 ADS8688 fuera de [-40^°C, +130^°C] o tensión fuera de [0.01V, 5.11V]).	Activa Failsafe Térmico: arranca ventilador de Motor al 10% y escala +10% cada segundo hasta 100% de potencia máxima continua.	Automática cuando ambos sensores NTC (Motor e Inversor) retornan valores válidos.	CAN ID 0x503 (Byte 0 = 1, Byte 3-4 = 201, Byte 5 = FanMotor%, Byte 6 = FanInv%)
202	FAULT_CODE_INV_NTC_FAIL	FAULT_CAT_HARDWARE	HIGH	≥ 3 lecturas fallidas consecutivas en sensor NTC Inversor (Canal 7 ADS8688).	Activa Failsafe Térmico: fuerza ventilador de Inversor al 100% mediante rampa de seguridad.	Automática tras restablecer lecturas válidas simultáneas.	CAN ID 0x503 (Byte 0 = 1, Byte 3-4 = 202)
203	FAULT_CODE_ADS8688_SPI_ERR	FAULT_CAT_HARDWARE	HIGH	Fallo de comunicación en bus SPI con el ADC ADS8688 o respuesta NOP no válida.	Invalida lecturas de temperatura y galgas STS; fuerza ambos ventiladores al 100% de potencia.	Requiere re-inicialización del periférico SPI o reinicio de placa.	CAN ID 0x503 (Byte 0 = 1, Byte 3-4 = 203)
204	FAULT_CODE_TWAI_BUS_OFF	FAULT_CAT_COMMUNICATION	HIGH	Controlador TWAI entra en estado TWAI_STATE_BUS_OFF por saturación de errores en bus físico.	Llama a twai_initiate_recovery(), reinicia el controlador TWAI y vacía la cola de transmisión.	Automática tras completar secuencia de recuperación del estándar CAN.	CAN ID 0x503 tras recuperar el bus

4. Matriz de Comunicación por Bus CAN / TWAI (500 kbps)

El bus CAN permite la interacción en tiempo real entre la ECU y el resto de subsistemas del monoplaza:

4.1. Tramas Transmitidas por la ECU

CAN ID	Nombre de Trama	DLC	Frecuencia	Destinatarios	Disposición de Bytes (Payload)	Factor de Escala y Unidades
0x401	ECU_TEMPS	4	1 Hz	MCU, Dashboard, Telemetría	Byte 0-1 : Temp Motor (Big-Endian int16) Byte 2-3 : Temp Inversor (Big-Endian int16)	1\ LSB = 1^°C (Rango: -40 a +130^°C)
0x402	ECU_STS_GAUGES	8	100 Hz	Telemetría, Data Logger, Dinámica	Byte 0-1 : STS Rear-Right (BE uint16) Byte 2-3 : STS Rear-Left (BE uint16) Byte 4-5 : STS Front-Right (BE uint16) Byte 6-7 : STS Front-Left (BE uint16)	1\ LSB = Raw ADC Count (0..65535)
0x503	ECU_DIAGNOSTIC_DTC	8	10 Hz / On-Fault	Safety Master, Data Logger, Dashboard	Byte 0 : Failsafe Activo (1 = Sí, 0 = No) Byte 1 : Categoría de Fallo (0=HW, 1=Comm, 2=Res, 3=Timing) Byte 2 : Prioridad (0=Low, 1=High) Byte 3-4 : Código DTC (uint16 BE) Byte 5 : Potencia Ventilador Motor (0..100\%)	Byte 6 : Potencia Ventilador Inversor (0..100\%) Byte 7 : Contador acumulado de fallos

4.2. Tramas Recibidas por la ECU

CAN ID	Emisor	Contenido Relevante	Acción Ejecutada en ECU
0x021	MCU (Motor Control Unit)	Byte 6 : Estado R2D del vehículo (4 = READY_TO_DRIVE).	Si el vehículo está en R2D, el servicio OTA (ota_service.c) bloquea inmediatamente cualquier intento de flasheo inalámbrico para impedir accidentes en pista.

5. Matriz de Validación de Pruebas Unitarias (Unity Test Suite)

Todas las funciones críticas han sido validadas en el entorno de pruebas unitarias x86 native de PlatformIO con el framework Unity:

Test Case	Archivo Fuente	Propósito de la Prueba	Aserciones Clave	Resultado
test_ads8688_bosch_ntc_conversion	test_main.cpp	Valida la tabla de interpolación Steinhart-Hart/Bosch a 20^°C (2500\ Ω), 50^°C (834\ Ω) y valores fuera de rango (0\ Ω \to NAN).	TEST_ASSERT_FLOAT_WITHIN(0.1f, 20.0f, t20) TEST_ASSERT_TRUE(isnan(t_invalid))	PASSED
test_fan_driver_esc_scaling	test_main.cpp	Verifica la conversión de porcentaje a pulsos μs (0%\ \to 1000μs, 50%\ \to 1570μs, 100%\ \to 2000μs) y cálculo de duty LEDC a 14 bits.	TEST_ASSERT_EQUAL_UINT16(1000, ...) TEST_ASSERT_EQUAL_UINT32(819, duty_min)	PASSED
test_fan_driver_slew_rate	test_main.cpp	Verifica que el limitador de pendiente restrinja el salto térmico a 2.0\% en 100 ms (20\%/s).	TEST_ASSERT_FLOAT_WITHIN(0.01f, 2.0f, current)	PASSED

Test Case	Archivo Fuente	Propósito de la Prueba	Aserciones Clave	Resultado
test_pid_cooling_reverse_action	test_main.cpp	Comprueba que el regulador PID en acción inversa dé salida 0% a temperatura fría (30^°C) y aumente la salida proporcionalmente al superar el setpoint (55^°C).	TEST_ASSERT_FLOAT_WITHIN(0.01f, 0.0f, out_cold) TEST_ASSERT_TRUE(out_hot > 20.0)	PASSED
test_fault_manager_failsafe_escalation	test_main.cpp	Valida el reporte de fallo crítico FAULT_CODE_MOTOR_NTC_FAIL , activación del enclavamiento de failsafe y generación del registro de diagnóstico.	TEST_ASSERT_TRUE(fault_manager_is_failsafe_active()) TEST_ASSERT_EQUAL_UINT32(201, rec.code)	PASSED

6. Conclusión de Paridad y Cumplimiento

El firmware ECU_FW supera con éxito todos los requisitos de calidad industrial y seguridad funcional:

- 100% de paridad funcional respecto a ecu.ino .
- Cero memoria dinámica (malloc , new , buffers dinámicos prohibidos).
- Determinismo FreeRTOS con tareas fijadas a núcleos dedicados y periodos exactos mediante vTaskDelayUntil .
- Validación continua en CI/CD: 100% de pruebas unitarias superadas sin advertencias de compilación (-Werror).